ONE THREAD · DENSO FACTORY HACKS 2026 · CHALLENGE D3

System architecture

How factory data travels from every department's systems to an AI agent that can trace a defect back to the machine, the lot and the documents behind it, and how the same data warns before, during and after a problem.

Version 2 · 10 November 2026 · nine-table data model, before / during / after early warning

Summary

Every department already keeps its data in its own databases: production in MES and the historian, the warehouse in WMS, quality in QMS, purchasing in ERP, design in PLM, and the rest in documents. They cannot be joined: the same machine, lot or supplier has a different name in each, the clocks disagree, and nobody knows which dataset holds what. One Thread fixes that without changing any of those systems, so a question can be answered across them with ordinary queries. Every record is mapped to one shared format, given one ID per business object (machine, lot, part, supplier, order, customer, person and so on) and placed on one plant clock. A linking step connects the records with a five-tier confidence ladder, from exact key matches down to suggestions that an engineer has to confirm. The result is a *digital thread*: a thin layer over the department data, made of nine tables that record what exists, what happened and how it is connected, plus a catalog that says where every dataset lives, whether copied into the lakehouse or left in its source system. The thread makes the department data joinable and findable; the department databases still hold the detail and answer most questions.

The same tables drive early warning at three moments. **Before** a problem, an LLM reads meeting minutes, notices and work orders for changes to man, machine, material or method, and opens a watch on everything the change can reach. **During** production, rules and simple models check live signals in that context. **After** an incident, the system traces back, finds similar past cases and stores the confirmed incident so the next one is caught earlier. An AI agent reads all of it through read-only MCP tools: it uses the graph to find which records matter, queries the department databases for the detail and the numbers, and searches the document store for text that no link points to. Every statement in its answers and reports links back to its source record.

Design principles

1. **Read-only at the source.** Connectors adapt to each system. No department has to change its software or send data in a new format.
2. **Keep the raw copy.** Everything is stored as received, so the thread can be rebuilt when mappings or link rules improve.
3. **Normalise before linking.** Shape, identity and time are fixed in three explicit steps, each driven by a versioned registry.
4. **Every link and every alert carries its evidence.** It records which rule made it, how confident it is and whether a person confirmed it.
5. **Copy what is used often, point to the rest.** Heavy or rarely used data stays in its source; the catalog knows where it is.
6. **The graph finds, the databases answer.** Links and the catalog decide which records and datasets a question needs; the numbers come from queries on the department data.
7. **The AI only reads and always cites.** The agent works through read-only tools and cannot assert anything it cannot point to.
8. **People approve, the AI suggests.** Mappings, aliases, weak links and incidents are proposed by the system and approved by an engineer.

Contents

1. Architecture diagrams
2. Layer by layer
3. Control plane
4. Unified data model
5. Before, during and after a problem
6. The AI agent and its tools
7. Tracing a customer claim three months later
8. Demo slice and target architecture
9. Technology summary
10. Open decisions
11. References
12. Appendix. How the two team drafts were merged

1 Architecture diagrams

Three diagrams describe the system at three levels. Figure 1 is the overview: data flows from the departments' systems, is normalised and linked into one digital thread, and is read by the AI agent and the dashboards through one query layer. Figure 2 shows every layer and both data paths in detail. Figure 3 shows how the stored data is organised and how the agent reaches it.

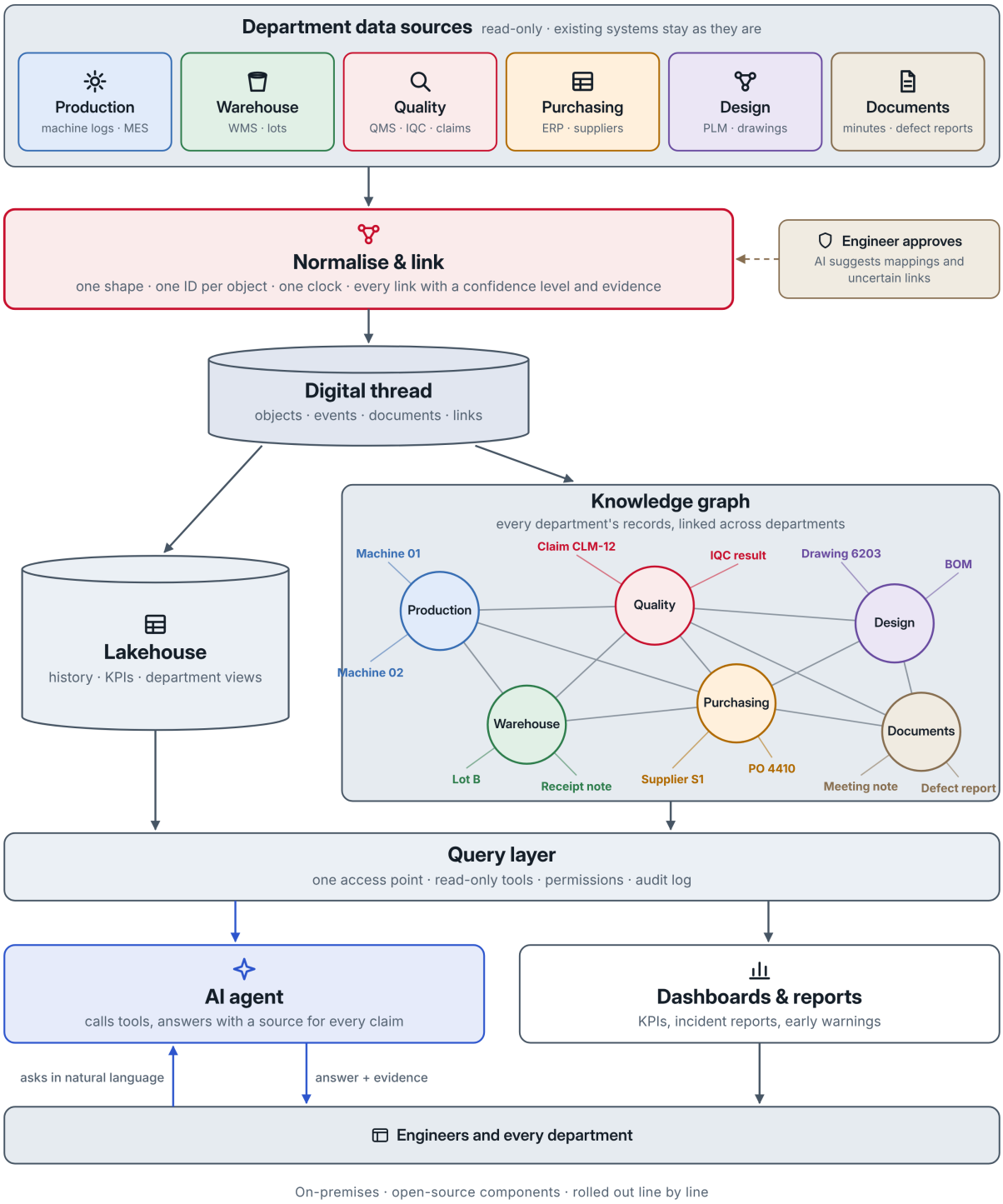


Figure 1. Overview. In the knowledge graph, each department's records hang off its node; a line between two departments means their records are linked.

Figure 2. Detailed pipeline

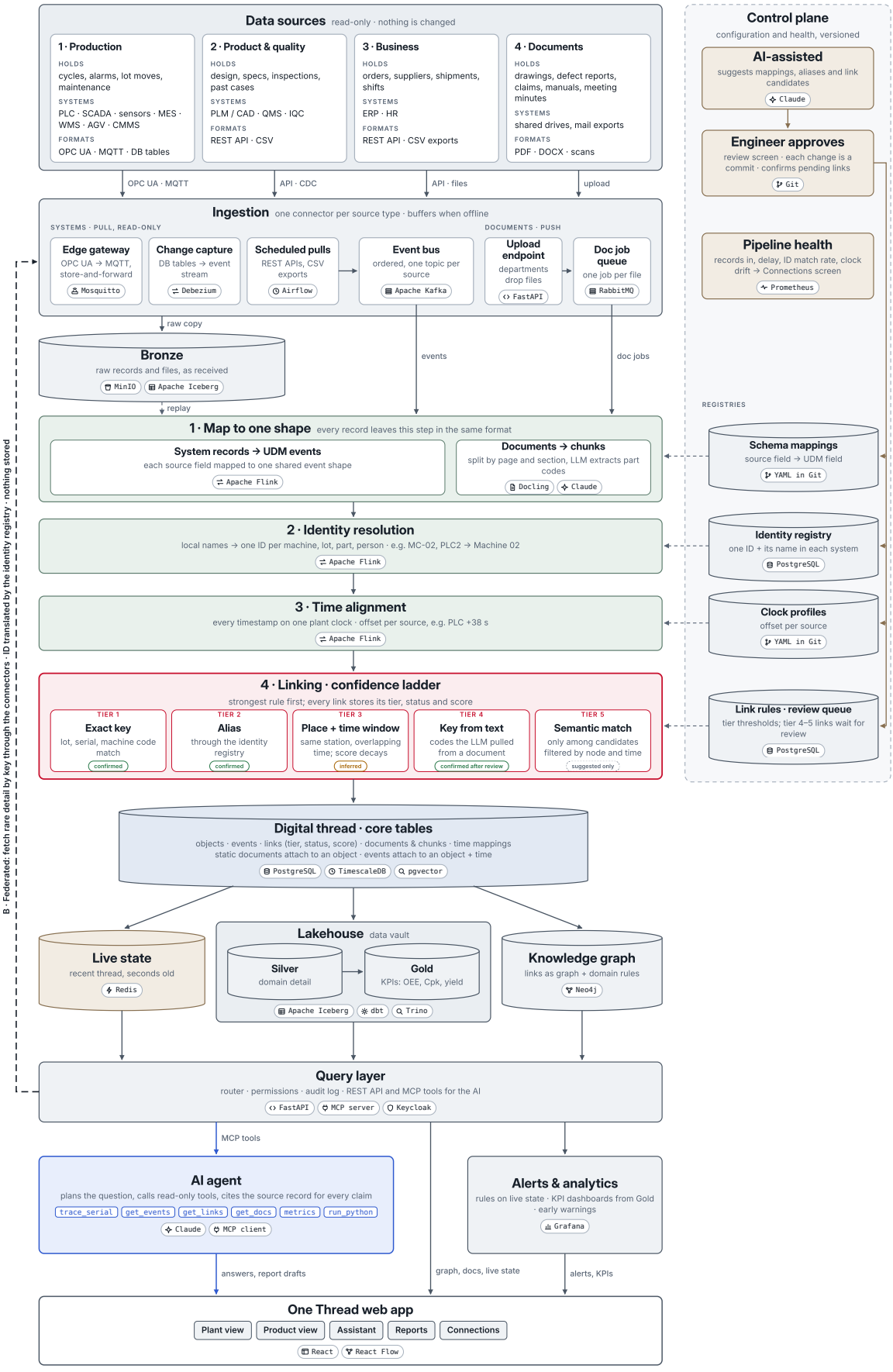


Figure 3. Where the data lives and how it is reached

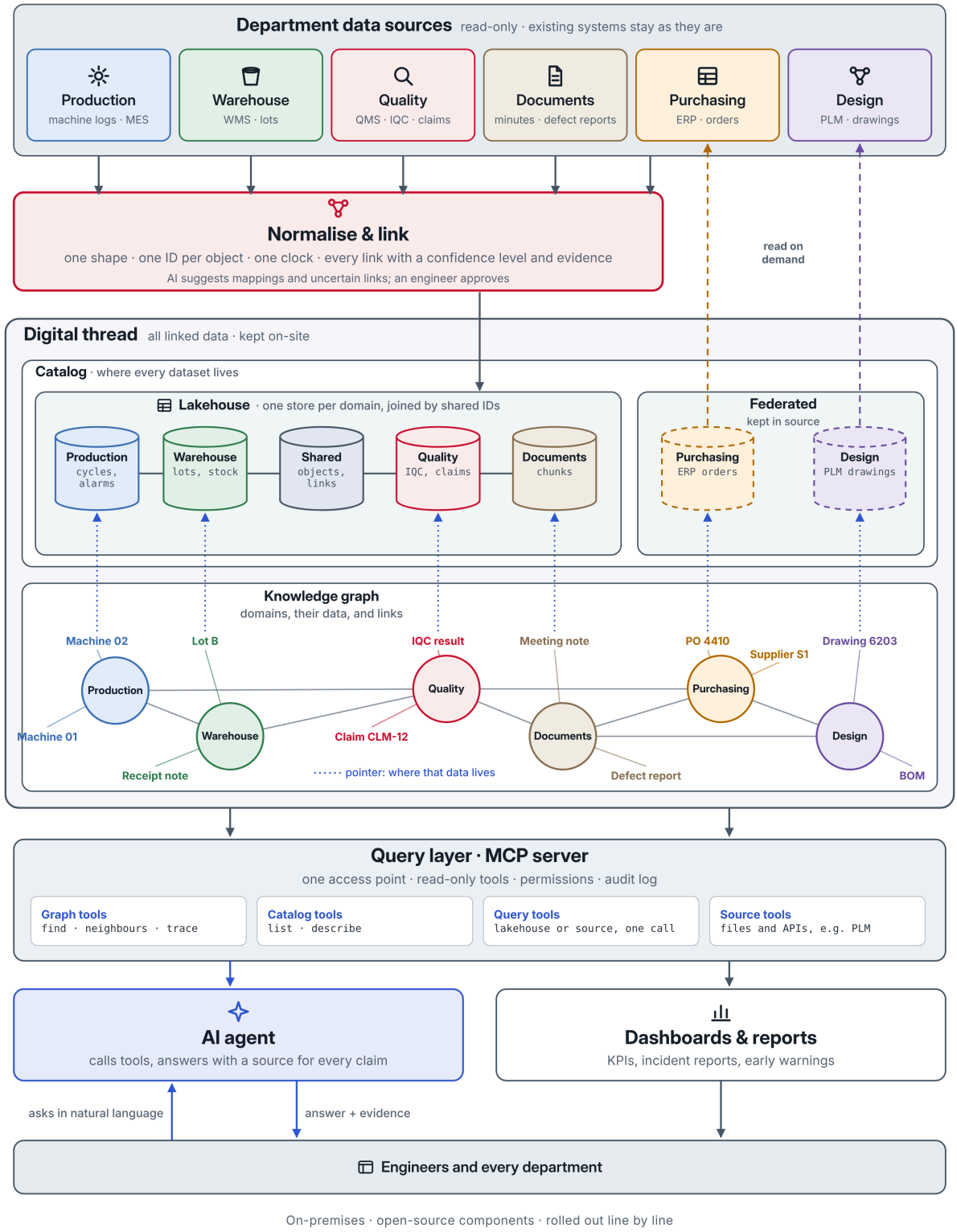


Figure 3. The digital thread holds a catalog and a knowledge graph. The catalog lists every dataset: domain stores in the lakehouse, joined by shared IDs, and federated stores that stay in their source and are read on demand. Each leaf in the graph points (blue dotted line) to where its data lives; the pointer is resolved through `aliases` and the catalog, not stored per object. The lines between department nodes are computed from the stored links. The agent and the dashboards reach both through the four tool groups of the MCP server.

2 Layer by layer

The running example is the demo story: Line 1 of the MH-20 water pump. On 22 October a meeting accepts bearing lot B from Supplier S1 as a temporary concession because of a shortage. Lot B is received on 28 October and loaded into Hopper 1 of Machine 02 on 29 October at 13:52. Cycle time rises from 30 s to 42 s and the noise-test fail rate from 0.5 % to 6 %. Lot B goes into 1,000 pumps. The noise test rejects about 60 of them; the other 940 pass and ship in 3 shipments. Some of those carry a latent defect: the bearing passes the end-of-line test and becomes noisy after weeks of use. A customer claim, CLM-12, arrives three months later.

2.1 Data sources

Six groups of systems hold the data. They are only read, never written to.

Production	PLC, SCADA, sensors, MES, AGV fleet, CMMS. Cycles, alarms, maintenance. OPC UA, MQTT, database tables.
Warehouse	WMS. Lots, stock, movements, hopper refills. Database tables.
Quality	QMS, incoming inspection (IQC). Inspection results, claims, past cases. REST API, CSV.
Purchasing	ERP. Purchase orders, suppliers, receipts, shipments. REST API, CSV exports.
Design	PLM / CAD. Drawings, BOMs, specifications, revisions. REST API, nightly export.
Documents	Meeting minutes, defect reports, change notices, machine manuals, HR shift rosters. PDF, DOCX, scans.

2.2 Ingestion

Two doors, one for systems and one for documents.

Systems are pulled by one connector per source type. An **edge gateway** on the shop floor reads PLCs over OPC UA and publishes over MQTT; if the network drops it stores messages and forwards them later. **Change capture** reads new rows from MES and WMS tables through the database log, so the source systems carry no extra load. **Scheduled pulls** fetch REST APIs and CSV exports, such as the nightly PLM export. All system data lands on one **event bus** with one ordered topic per source.

Documents are pushed. Departments drop files into an **upload endpoint**; each file becomes a job on a **document queue**. This is the one place where a push endpoint makes sense, because documents have no system to pull from.

How each source is read, with rough rates for planning:

SOURCE	HOW IT IS READ	HOW OFTEN (ESTIMATE)
PLC, sensors	OPC UA through the edge gateway, either directly from the PLCs or from the SCADA / historian that already collects them	Streamed; one value per cycle to one per second
SCADA, historian	OPC UA or the historian's API	Streamed, or batches every minute
MES	Change capture on its database, or its API	Seconds
WMS	Change capture on its database, or its API	Seconds
CMMS	API or export	Hourly
QMS, IQC	REST API or CSV export	Hourly; claims as they are logged
ERP	API for orders, receipts and suppliers; order detail read on demand (federated)	Every 15–30 min; detail on request
PLM, CAD	Nightly export of BOMs and revisions; drawings read on demand (federated)	Nightly; files on request
HR rosters	CSV export	Per shift
Documents	Upload door	When filed

TECH Mosquitto (MQTT), OPC UA, Debezium, Airflow, Apache Kafka, FastAPI, RabbitMQ

2.3 The raw copy (Bronze)

Every event and every file is stored exactly as received, with its arrival time and source.

Bronze is never edited. Its purpose is **replay**: when an engineer approves a new alias (for example that the supplier code "S1-6203-B" means Bearing 6203 rev B), the affected history can be re-processed and old records gain links they could not get before.

TECH MinIO (on-premises S3 storage), Apache Iceberg tables in Parquet

2.4 Map to one shape (step 1)

Every record leaves this step in the same format.

System records are mapped field by field to the unified data model (UDM, section 4): which objects, what happened, when, from which source record. Measured values such as cycle times become **samples** of a signal; business records such as a goods receipt become **events**. The mapping for each source lives in the schema-mapping registry, so adding a source means adding a mapping file, not new code.

Documents are parsed and split into chunks by page and section. An LLM reads each chunk and extracts the codes it mentions (part numbers, lots, machine names) and any change it describes. A 40-page MH-20 design file becomes, for example, one chunk for the housing drawing, one for the bearing and one for the seal.

Every chunk gets **metadata** whether or not it links to anything: department, document type, date, plant and line, process, language and source. A document with no codes at all, such as a lessons-learned note or a general SOP, is still stored, embedded and searchable by that metadata (section 6). Links are added when the chunk names something; they make retrieval more precise but are not needed to find the document.

TECH Apache Flink jobs on the Kafka streams; Docling for parsing; Claude for code and change extraction

---

2.5 Identity resolution (step 2)

Local names become one ID per business object, whichever department owns it.

MES writes "MC-02", the PLC sends "PLC2" and maintenance calls it "Press 2". The `aliases` table holds one row per name per system, so all three resolve to `Machine 02`. Each alias is valid for a period: if a PLC is replaced and its tag names are reused on another machine, old data still resolves to the old machine. Sensor tag names such as `PLC2.CT` are aliases too: they resolve to a signal (Machine 02, cycle time). Names that match nothing are held in quarantine and appear on the Connections screen for review.

TECH Flink lookup against the aliases table in PostgreSQL

2.6 Time alignment (step 3)

Every timestamp is moved onto one plant clock.

Each source has a row in `time_mappings`: its offset, how uncertain it is and since when. Machine 02's PLC runs 38 seconds fast, so 13:52:38 on the PLC becomes 13:52:00 plant time, the same moment WMS recorded the hopper refill. Without this step the time-window rule in step 4 would link the wrong records, and the alert in section 5 would point at the wrong lot.

TECH Flink, with offsets measured from known shared events such as a lot scan

2.7 Linking with a confidence ladder (step 4)

Records are connected to objects and to each other, strongest rule first. This is where the solution's value sits.

TIER	RULE	STATUS	EXAMPLE
1	Exact key: lot, serial, canonical machine code	confirmed	The WMS refill record names lot B and Hopper 1 of Machine 02; the MES run log names serial 0418 and Machine 02.
2	Alias through the aliases table	confirmed	"MC-02" and "PLC2" become Machine 02, then link as in tier 1.
3	Same station and overlapping time window	inferred	Lot to serial: when lot B was added at 13:52, a few lot A bearings were still in the hopper. Serials made from 13:52 to 13:56 link to both lots with a lower score; serials made later, such as 0418 at 14:05, link to lot B with a high score. A vibration log with no serial links the same way to the parts made in its window.
4	Key extracted from text by the LLM	confirmed after review	The minutes of 22 October mention "lô B"; the link to lot B is used once an engineer accepts it.
5	Semantic similarity, only inside candidates already filtered by object type and time	suggested only	An old defect report about "bearing noise" with no part code is shown as a possible link to Bearing 6203. It is never used as fact.

Every link stores its tier, status, score, the rule (condition) and version that made it, the records used as evidence, and the time it was valid: lot B fed Machine 02 from 13:52 on 29 October until the lot ran out. The mixing window comes from the hopper size and the cycle time, and is set per station in the link rules. Tier 4 and 5 links wait in the review queue until an engineer decides. Figure 5 in section 4 shows how these rows become the edges of the knowledge graph.

TECH Flink for tiers 1–3, Claude for tier 4 extraction, pgvector for tier 5 similarity; rules and the review queue in PostgreSQL

2.8 Checking against the expected map

Every record and every link is checked against a map of what should exist, kept at the level of systems and object types.

The **expected map** combines the source and dataset catalogs with the type and relation catalogue (section 3). It says which systems feed which object types, which relations are allowed between which types, and which links every record of a type should end up with. It is configuration, written and approved by people, so it is stored. An excerpt:

FROM (SYSTEM)	RELATION	TO (SYSTEM)	EXPECTED
lot (WMS)	`received_under`	purchase_order (ERP)	every lot
lot (WMS)	`fed_into`	machine (MES, PLC)	every lot_in_use event
serial (MES)	`processed_on`	machine (MES)	exactly one per station
claim (QMS)	`concerns`	serial (MES)	every claim
document chunk	`mentions`	any object type	no target; reviewed

CHECK	WHEN	EXAMPLE OF A FAILURE	WHAT HAPPENS
Known source and dataset	Step 1	Messages arrive from an unregistered topic "PLC7"	Held in quarantine; shown on the Connections screen
Shape	Step 1	A WMS movement has no lot code	Row rejected and counted in pipeline health
Type allowed for the system	Step 2	A WMS name resolves to a supplier	Alias held for review
Relation allowed between the types	Step 4	A rule tries to link lot B `fed_into` Supplier S1	Link rejected; the rule is flagged
Time and count rules	Step 4	Lot B fed into a machine before its goods receipt; a serial on two machines at one station	Conflict sent to the review queue
Coverage	Daily	3 lots in use have no receipt link to ERP	Health metric and a review item per gap

The department-level lines in the knowledge graph are the **observed map**: they are computed from the stored links (how many links join each pair of departments, and how many are confirmed), not stored themselves. Comparing the observed map with the expected one shows gaps. A thin or missing line where one is expected usually means a connector, mapping or alias problem, not a factory problem, and it is caught before anyone relies on a trace.

TECH Checks in the Flink jobs (steps 1–4) and a daily dbt test suite for coverage; results in Prometheus

2.9 The digital thread

The store of record. Everything downstream is built from here.

The thread is not a second copy of the factory that is walked object by object. It is the layer that makes the department databases work together: shared IDs, links that act as join tables between departments, a catalog of where every dataset lives, and the events that give the data its context. It has two parts (Figure 3). The **catalog** lists every dataset and where it lives. Data that is used often is copied into the **lakehouse**, one store per domain (production, warehouse, quality, documents, and a shared store), joined by the shared IDs. Data that is heavy or rarely needed, such as ERP order detail or PLM drawings, stays in its source as a **federated** store and is read on demand. The **knowledge graph** sits on top: its nodes are business objects and events, its edges are the links, and each leaf points to the dataset that holds its detail.

The catalog has two levels. **Sources** has one row per system: its domain, connector, whether its data is copied or federated, where it lands and who owns it. **Datasets** lists what each source holds: the object types, the location (a lakehouse table or a connector endpoint) and the key field. No pointer is stored per object. A pointer is resolved when needed: PO 4410 → `aliases` → ERP, key 4500004410 → catalog → `erp.purchase_orders`, federated, keyed on po_number → the query layer calls the ERP connector. Moving a dataset from federated to copied changes one catalog row and nothing in the data.

The nine tables of the unified data model (section 4) live in the shared store. They are a thin core that records what exists, what happened and how it is connected; section 4 lists which store owns which detail. Sample values are not in them: they stay in the time-series store or the source, and the `signals` table says where.

TECH PostgreSQL for the nine tables, TimescaleDB for samples, pgvector for chunk embeddings, Apache Iceberg for the domain stores

2.10 Read-optimised views

Live state	The last few hours of the thread, seconds old. Feeds Plant view: machine status, edge speeds, what is feeding which machine. `Redis`
Sample store	Signal samples keyed by `signal_id`, already on plant time. Used by the detectors and the charts. `TimescaleDB`
Lakehouse	**Silver** keeps clean domain detail for the full history. **Gold** holds the business views and KPIs such as OEE, Cpk and yield. `Apache Iceberg on MinIO, dbt, Trino`
Knowledge graph	Objects, events and links as a graph, plus domain rules such as "a lot used at a station reaches every part processed there while the link was valid". Answers multi-hop questions fast: lot → parts → boxes → shipments. The department-level lines in Figures 1 and 3 are computed from the links, not stored (section 2.8). `Neo4j`

2.11 Query layer

One door for every screen and for the AI.

The query layer is an **MCP server** with four tool groups, also exposed as a REST API for the web app. Most requests are database queries; the graph is used to decide what to query (section 6 shows the patterns). **Catalog tools** list the datasets for a domain and describe one: fields, owner, freshness and where it lives. **Graph tools** find an object, list its neighbours and trace forward or backward. **Query tools** run one call against the lakehouse or a federated source, read events and samples in a time window, search the document store and find similar past cases. **Source tools** fetch files and call source APIs, for example a drawing from PLM. The full list is in section 6.

Permissions decide who sees what, which matters because HR, cost and supplier data are now one query away. An **audit log** records every read, including the agent's. For a **federated** read, the query layer translates the canonical ID back to the source system's name through the aliases table, finds the dataset and connector in the catalog, and fetches the data. Nothing is stored.

TECH FastAPI, MCP server, Trino for federated SQL, Keycloak for sign-in and roles

2.12 Applications

AI agent	Plans the question, calls read-only tools, writes answers and report drafts in which every claim cites its record. Runs on request, on events and on a schedule. See section 6.
Early warning	Detectors for before, during and after a problem write their findings as events into one review queue (section 5). KPI dashboards read Gold. `Grafana`
Web app	Plant view, Product view, Assistant, Reports and Connections. Graph screens use a node-and-edge canvas. `React, React Flow`

3 Control plane

The control plane holds everything that configures the pipeline and shows whether it is healthy. Its loop is the same for every kind of change:

1. **AI suggests.** An LLM proposes mappings for new source fields, aliases for unmatched names ("PLC3 is probably Machine 03"), candidate links from tiers 4 and 5, and changes it found in documents.
2. **Engineer approves.** A review screen shows each suggestion with its evidence. Approving writes a commit or an annotation, so every change has an author and a date and can be rolled back.
3. **Registries update.** The pipeline reads the new version; Bronze replay applies it to history when needed.

Some registries are configuration files; others are tables of the data model, because the records need to point at them. The source catalog, dataset catalog and type and relation catalogue together form the expected map used for validation (section 2.8).

REGISTRY	HOLDS	USED BY	KEPT IN
Schema mappings	Source field → UDM field, per source	Step 1	YAML in Git
Source catalog	One row per system: domain, connector, copied or federated, where it lands, owner, access role	Ingestion, step 1, query layer, Connections screen	YAML in Git
Dataset catalog	What each source holds: object types, location, key field, freshness	Step 1, query layer, catalog tools	Generated from the schema mappings; PostgreSQL
Type and relation catalogue	Allowed object and event types, the department of each type, allowed relations between types, expected coverage, field checks per type	Steps 1–4, expected-map checks, AI agent	YAML in Git
Identity registry	One canonical ID and its name in each system, including tag names	Step 2, federated reads	`aliases` table
Signal catalogue	Each measured stream: object, measure, unit, where the samples live	Step 1, detectors, charts	`signals` table
Clock profiles	Offset and uncertainty per source, with valid-from date	Step 3	`time_mappings` table
Link rules, detectors, watch rules, extractors	Every rule or model that creates links or events, with its version and parameters	Step 4, section 5	`conditions` table; code in Git
Review queue	Pending links and alerts, each with an owner, and every decision on them	Step 4, section 5	`links` (pending) and `annotations`

Pipeline health measures records in, delay per source, ID match rate, clock drift and coverage of expected links. These are the numbers on the Connections screen ("Matched to one ID 99.2 %", "ERP 30 min delayed"), so the data team knows the thread is complete before anyone trusts an answer.

`Prometheus`

4 Unified data model

The unified data model (UDM) is the index and join layer over the department data, not a replacement for it. It is deliberately generic: nine tables that stay the same whichever department or use case is added. A new department adds **rows** (object types, relation names, conditions, attributes), not new core tables. The tables come in three groups:

- **Things that exist.** `objects`, the names each system gives them (`aliases`), and the streams measured on them (`signals`).
- **What happened.** `events`, which can be a moment or an interval; the `conditions` (rules, models, mappings, LLM extractors) that produced them; and what people said about them (`annotations`).
- **How it connects.** `links` between any two records, the `documents` they point to, and the `time_mappings` that put every clock on plant time.

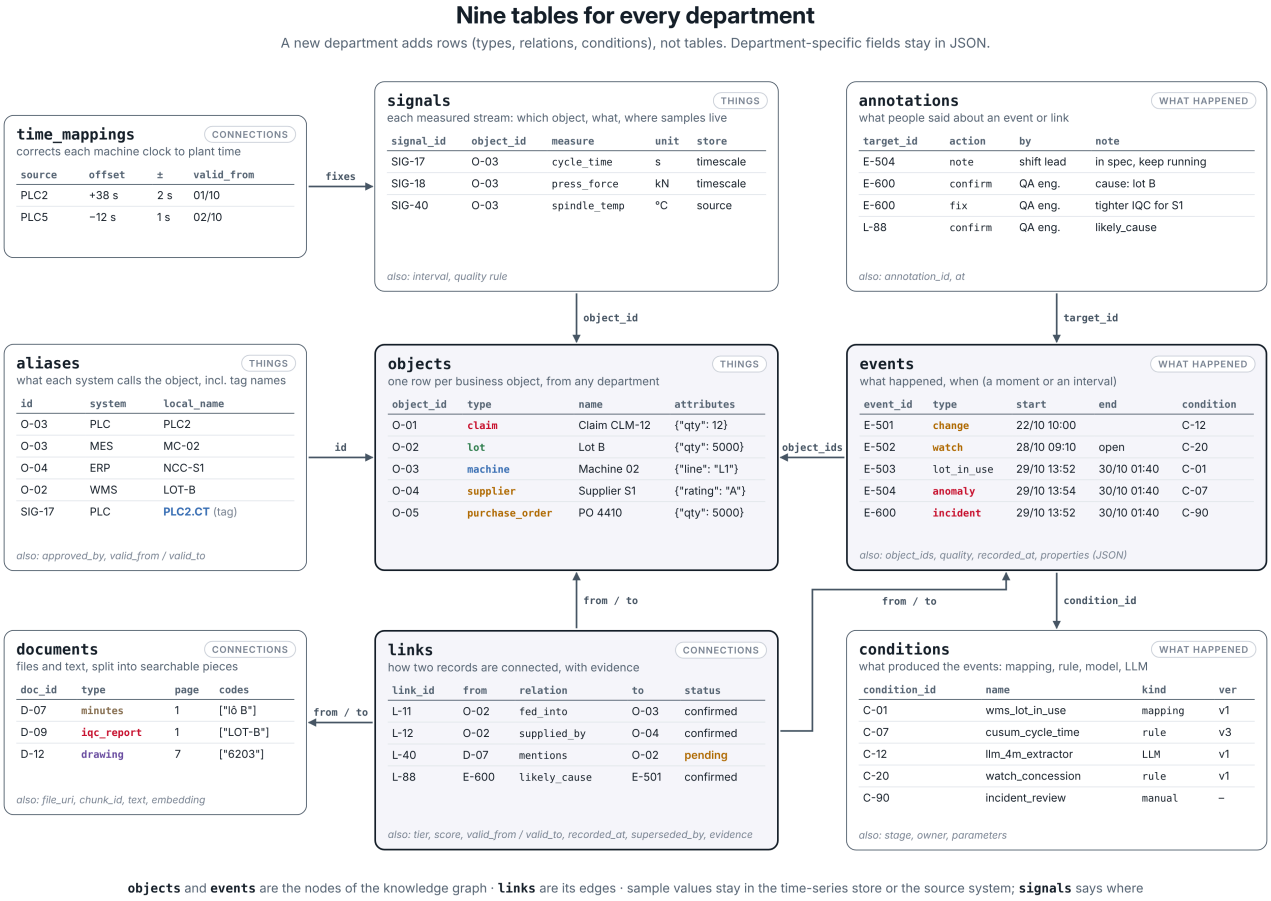


Figure 4. The nine tables with rows from the demo story. Arrows show which field points to which table. The full field lists follow.

TABLE	MAIN FIELDS	EXAMPLE ROWS
`objects`	object_id, type, name, attributes (JSON)	Machine 02 (machine); Lot B (lot); Supplier S1 (supplier); PO 4410 (purchase_order); Claim CLM-12 (claim)
`aliases`	id, system, local_name, approved_by, valid_from, valid_to	Machine 02 ← "PLC2" in PLC, "MC-02" in MES; signal SIG-17 ← tag "PLC2.CT"

TABLE	MAIN FIELDS	EXAMPLE ROWS
`signals`	signal_id, object_id, measure, unit, store, interval, quality rule	SIG-17: Machine 02 cycle time, seconds, TimescaleDB; SIG-40: spindle temperature, left in the source
`events`	event_id, type, object_ids, start, end (empty for a moment), time_kind, condition_id, quality, source_record_id, recorded_at, superseded_by, properties (JSON)	E-501 change, 22/10; E-503 lot_in_use 29/10 13:52 → 30/10 01:40; E-504 anomaly from 13:54, with its feature vector in properties
`conditions`	condition_id, name, kind (mapping, rule, model, LLM, manual), version, stage, owner, parameters	C-01 wms_lot_in_use; C-07 cusum_cycle_time v3; C-12 llm_4m_extractor; C-20 watch_concession
`annotations`	annotation_id, target_id, action (flag, confirm, false alarm, note, label, fix), by, at, note	Shift lead on E-504: "in spec, keep running"; QA engineer on E-600: confirm, fix "tighter IQC for S1"
`links`	link_id, from_id, to_id, relation, tier, status, score, condition_id, valid_from, valid_to, recorded_at, superseded_by, evidence	Lot B fed_into Machine 02, tier 1, valid 13:52 → 01:40; lot B consumed_in serial 0418, tier 3, score 0.98; minutes mention lot B, tier 4, pending; E-600 likely_cause E-501
`documents`	doc_id, file_uri, type, chunk_id, page, text, extracted_codes, embedding, department, doc_date, plant, line, process, language	Minutes of 22/10 mentioning lot B; IQC report for lot B; drawing 6203, page 7
`time_mappings`	source, offset, uncertainty, valid_from	PLC2, +38 s, ±2 s, since 1/10

Events as moments and intervals

An event with only a start is a moment: a goods receipt, a claim. An event with a start and an end is an interval: a lot in use, a watch, an anomaly, an incident. Intervals make the questions in section 5 simple: "did the anomaly overlap the time lot B was in use?" is one overlap query. Every event records the condition that produced it, so a rule change or a new model version can be traced and its alerts compared. `time_kind` says what the timestamp means: *sensor* (a machine clock, corrected in step 3), *transaction* (when a business record was posted) or *effective* (when a drawing, price or rule starts to apply).

Two kinds of time

Valid time (`start` / `end` on events, `valid_from` / `valid_to` on links and aliases) says when something was true in the factory. **Recorded time** (`recorded_at`) says when the system learned it. A correction never overwrites: the new row points to the old one through `superseded_by`, and a rejected link is marked rejected, not deleted. This answers both "what was true at 14:05?" and "what did the screen show at 13:54?", which matters when a claim or an audit asks why a decision was made.

Mixed references. `from_id` and `to_id` can point to an object, an event or a document, which a database foreign key cannot check. IDs therefore carry a prefix (O- for objects, E- for events, D- for documents, SIG- for signals), and a nightly check reports any link whose ends do not exist.

Links become the knowledge graph

Figure 5 shows how the `links` table becomes the knowledge graph: every row is one edge, links still waiting for review are kept apart from confirmed ones, and the valid time says when the edge held.

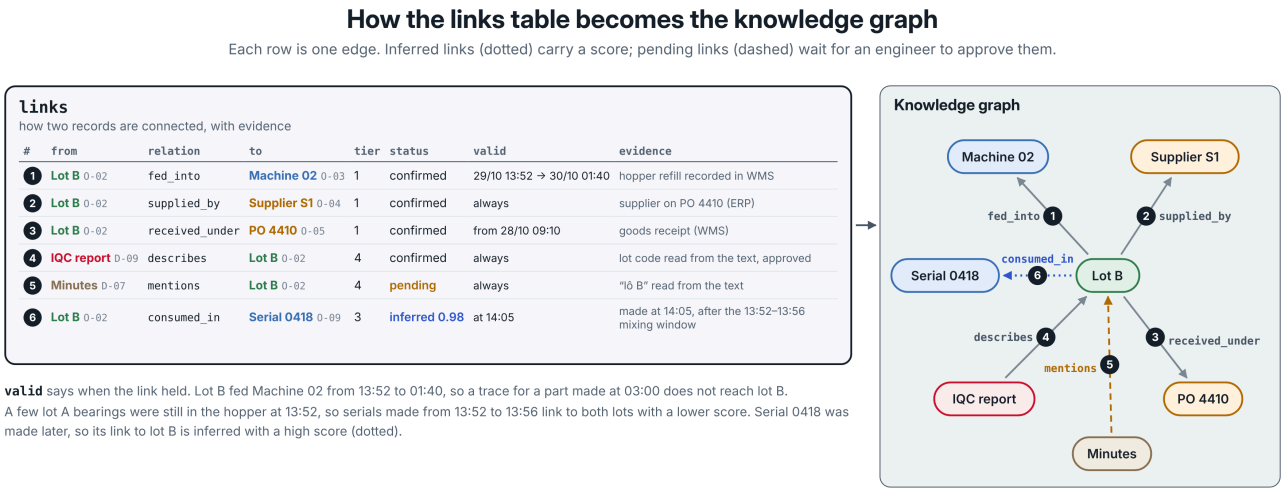


Figure 5. Each numbered row of `links` is the edge with the same number in the graph. The dotted link (tier 3, inferred) carries a score. The dashed link (tier 4, pending) is shown to users as a lead, not as fact, until an engineer approves it.

Which store holds what

Each fact has one owner. The nine tables are a thin core: IDs, names, events with the few attributes needed to link and filter them, links, and decisions. Full detail stays in the domain store or in the source, and is reached through the catalog. Nothing is kept in two places as a second source of truth; the other stores are either the owner of their detail or a copy that can be rebuilt.

STORE	OWNS	EXAMPLE FROM THE STORY
Nine core tables	Objects, aliases, signals, events, conditions, annotations, links, document chunks, clock offsets	Goods receipt event: lot B, PO 4410, quantity, time
Domain stores (Silver)	Clean detail per department	IQC measurements for lot B; MES run log lines
Sample store	Signal samples on plant time, keyed by `signal_id`	Every cycle time of Machine 02
Federated sources	Detail left in ERP, PLM and similar systems	Order lines and prices of PO 4410; drawing 6203
Bronze	The raw copy, as received	The original WMS refill message
Gold	KPIs and reports (rebuilt)	Cpk per supplier lot, OEE per shift
Knowledge graph, vector index	Nothing; both are projections rebuilt from the core	The trace from serial 0418 to Supplier S1

How the model grows without changing

- **Types, relations and conditions are rows or configuration.** Adding Purchasing means adding the types `supplier` and `purchase_order`, a mapping file and, if needed, a new condition.
- **Department-specific fields stay in JSON.** A supplier's tax code, a machine's model and a lot's expiry date sit in `attributes` or `properties`. An optional check per type validates the fields that matter.
- **A field becomes a shared type only when departments link through it.** A supplier is a type because Purchasing, Quality and the warehouse all refer to it. A machine's paint colour stays an attribute.
- **Department-shaped tables are built downstream.** Silver and Gold produce tables such as a purchasing view or a quality view, so the core stays generic and reports stay easy to query.

- **The core stays small, not frozen.** High-volume tables (events, links) are partitioned by time, indexes are added as query patterns settle, and a few typed columns may be promoted out of JSON when they are filtered on constantly. What does not happen is a new core table per department.

DEPARTMENT	SHARED OBJECT TYPES (STARTING SET)
Production	machine, line, station, part, serial
Warehouse	lot, location, box
Design	product, part number, drawing revision, BOM
Purchasing	supplier, purchase order, goods receipt
Quality	inspection, claim, defect report
Maintenance	work order, spare part
HR	employee, shift, qualification
Sales, logistics	customer, sales order, shipment
All	meeting, decision

Relations come in four groups. **Product flow:** `part_of`, `next_station`, `fed_into`, `consumed_in`, `processed_on`, `packed_in`, `shipped_in`. **Business:** `supplied_by`, `received_under`, `concerns`, `ordered_in`, `delivered_to`, `performed_on`, `operated_by`, `specified_by`, `revision_of`. **Knowledge:** `describes`, `mentions`, `decided_in`. **Incidents:** `watches`, `overlaps`, `likely_cause`, `similar_to`. New names go through the same suggest-and-approve flow as every other registry.

5 Before, during and after a problem

Early warning is organised by when it acts, not by which algorithm it uses. **Before** a problem, it watches changes that are written down. **During** production, it checks live signals in the context of those changes. **After** an incident, it traces back, finds similar past cases and keeps what was learned. All three use the same nine tables, and they feed each other: a confirmed incident becomes a better watch rule for next time.

Every detector, whether a rule, a model or an LLM extractor, is a **condition** that writes **events**, usually intervals. All of them land in one review queue, where people confirm, dismiss or comment, and each decision is an **annotation**. This is the same pattern that industrial analytics tools such as Seeq use (signals, conditions and capsules) [8]; One Thread adds links across departments and documents.

STAGE	TRIGGERED BY	METHODS	MAIN TABLES WRITTEN	IN THE DEMO
Before	New minutes, notices, work orders, change requests, receipts	LLM 4M extraction; rules for known risky changes; forward trace	`events` (change, watch), `links`, `documents`	Yes
During	New samples and business events	Limits and SPC; CUSUM; baseline model; PCA T^2 / SPE; Isolation Forest; overlap and time-slice queries	`events` (lot_in_use, anomaly), `links`, `annotations`	Limits, CUSUM, baseline
After	Claim, confirmed alert, weekly schedule	Backward trace; similar past cases (k-nearest neighbours + TF-IDF); recurring-pattern scan	`events` (incident), `links`, `annotations`, `conditions`	Yes, on a simulated incident library

Six kinds of question cover almost everything the stages need, and each maps to one tool (section 6):

QUERY	ASKS	EXAMPLE
Aggregate	How does this compare across lots, shifts or suppliers?	Noise-test fail rate by supplier lot in October, from the Quality and Warehouse stores
Forward trace	What can this reach?	Lot B → machines that will use it → products → shipments
Backward trace	Where did this come from?	Returned serial 0418 → Machine 02 at 14:05 → lot B → Supplier S1
Time slice	What happened here in this window?	All events on Machine 02 between 13:30 and 14:00
Overlap	Which intervals overlap this one?	Anomaly E-504 overlaps lot_in_use E-503 and watch E-502
Similarity	Have we seen this before?	E-504 is closest to I-0031, bearing noise from S1 in August

Before a problem

Most quality problems start with a change to man, machine, material or method (4M), and that change is usually written down somewhere: in meeting minutes, a change notice, a maintenance work order or a shift roster. An LLM extractor reads each new document chunk and writes a **change** event with the objects it concerns. Rules cover changes that are already structured, such as a concession lot, a new operator on a station or a new drawing revision. From the changed object, a forward trace finds everything it can reach, and a watch rule opens a **watch** interval with tighter limits on the signals involved. Figure 6 shows the steps for lot B.

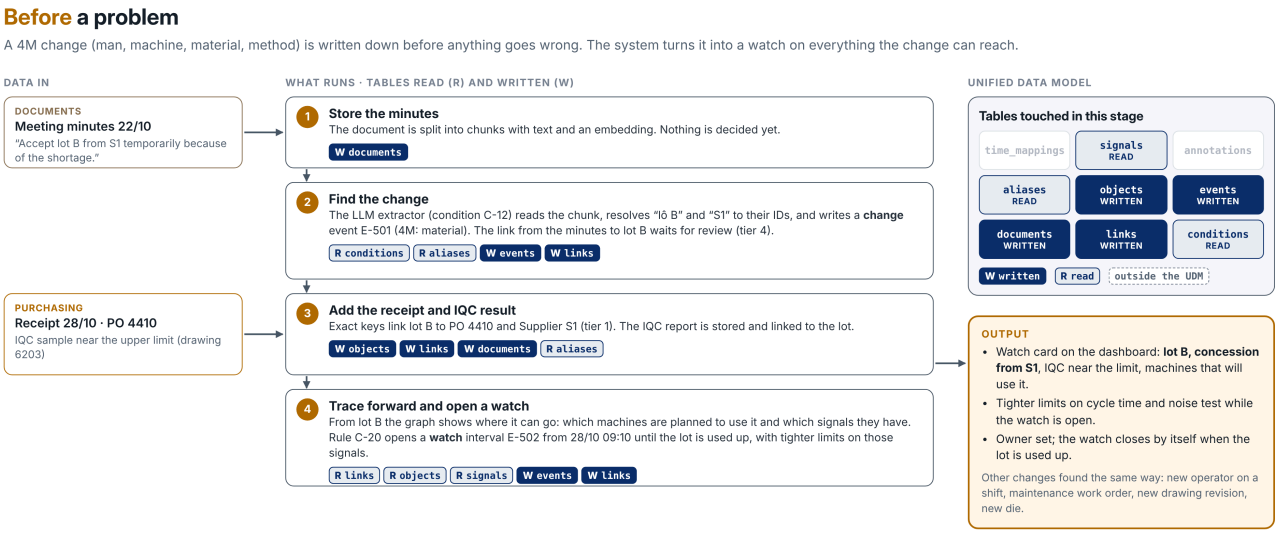


Figure 6. Before. Each step shows the tables it reads (R) and writes (W); the grid on the right sums them up for the stage.

Honest limits. LLMs out of the box are not reliable classifiers of maintenance and failure text: one study found an F1 score of 0.46 for GPT-3.5 on failure-mode classification, rising to 0.80 after fine-tuning on labelled examples [7]. That is why extracted changes are tier 4: they are shown at once as leads and used as fact only after review, and the reviewed examples are kept to improve the extractor.

During production

Samples arrive on plant time and are checked by simple, well-known methods. **Limits and SPC rules** catch values outside the normal band [2]. **CUSUM** adds up small deviations so a steady shift raises an alert before any single point crosses a limit [1]. A **baseline model** per machine (gradient-boosted trees [5, 6]) predicts the expected cycle time or force from product, shift, lot and temperature, so a value can be unusual for its context even inside fixed limits. **PCA** with T^2 and SPE statistics [3] and **Isolation Forest** [4] flag unusual combinations of several signals. Each finding is written as an **anomaly** interval whose properties hold a short feature vector: how much each signal shifted, for how long, and what changed just before.

Context then decides the rank. An overlap query checks whether the anomaly overlaps an open watch or a lot in use; a time-slice query lists what else happened on the machine in the window. An anomaly that overlaps a watch on a concession lot ranks above an isolated spike. Figure 7 shows the flow; Figures 8 and 9 show the methods on illustrative data.



Figure 7. During. Sample values stay outside the data model (dashed chip); only what they mean is written as events.

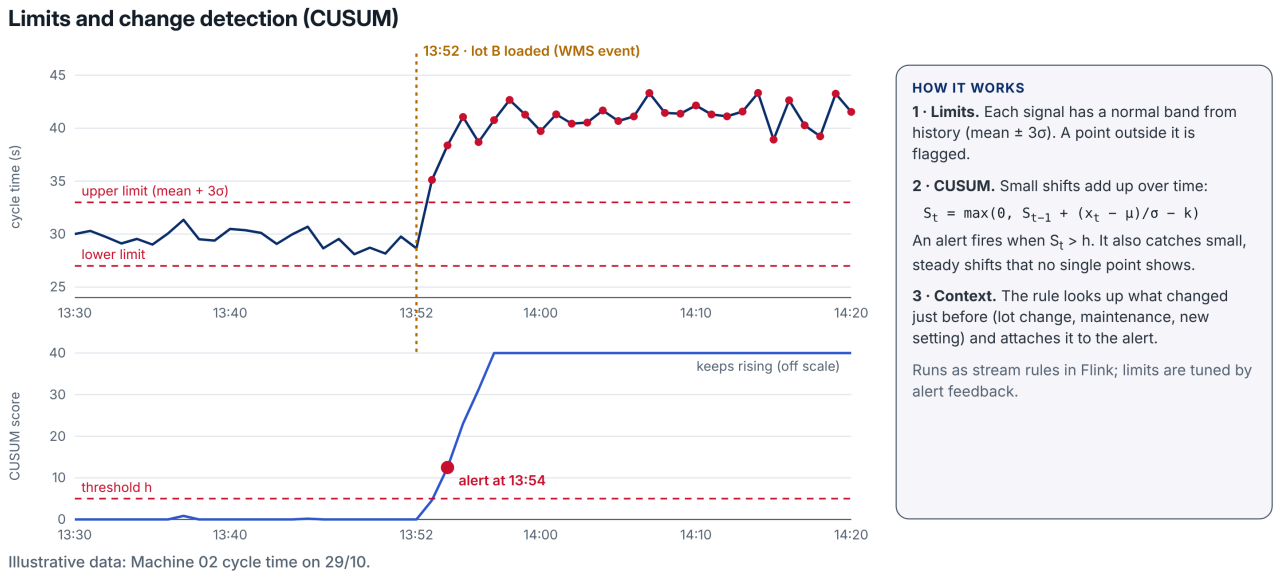


Figure 8. Limits and CUSUM on Machine 02's cycle time. The limit catches the jump; CUSUM also catches small steady shifts that no single point shows.

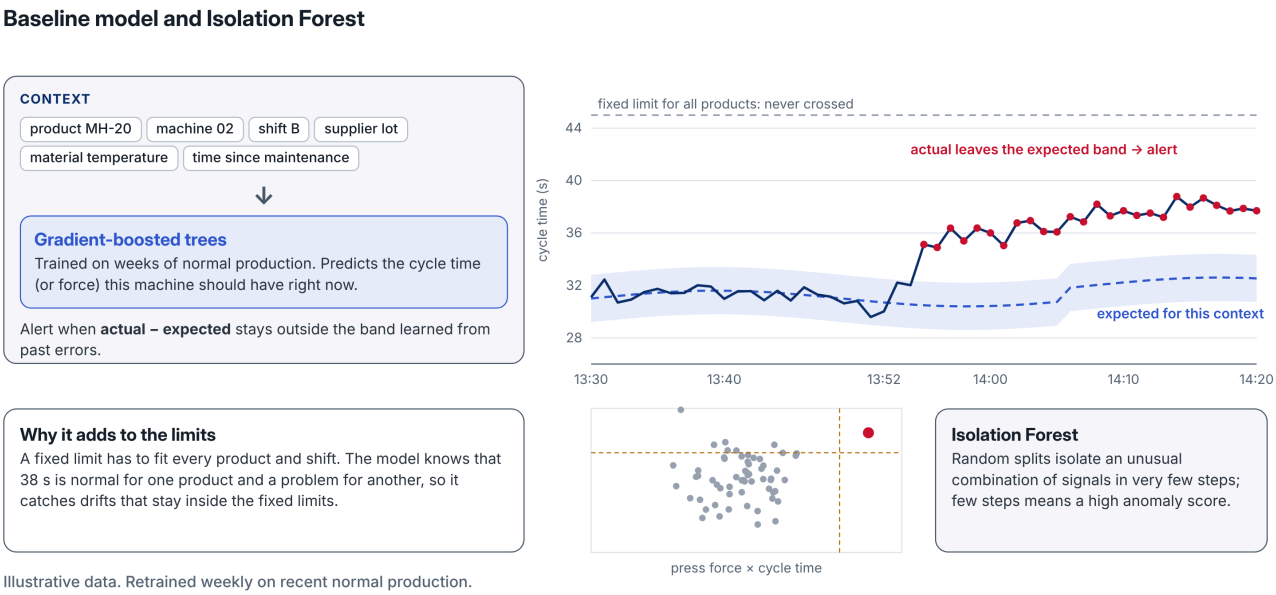


Figure 9. Baseline model and Isolation Forest. The model knows what is normal for this product and context, so it catches drifts that stay inside fixed limits.

After an incident

When a claim arrives or an engineer confirms an alert, a backward trace follows the links that were valid at the time of production back to the lot, the supplier and the documents. The system then looks for **similar past cases**. Each confirmed incident is kept as an interval event with a feature vector and its notes; a new case is compared with them by k-nearest neighbours on the standardised features and TF-IDF similarity on the text [9]. Nothing is trained on failures, so the few incidents a plant has are enough to start, and a case with no close neighbour is marked as a new kind of problem. When the engineer confirms the incident, the cause and the fix are stored as a link and annotations, and the watch rule that should have caught it gets a new version. A weekly job scans for repeats, such as the same supplier with the same signal shift.

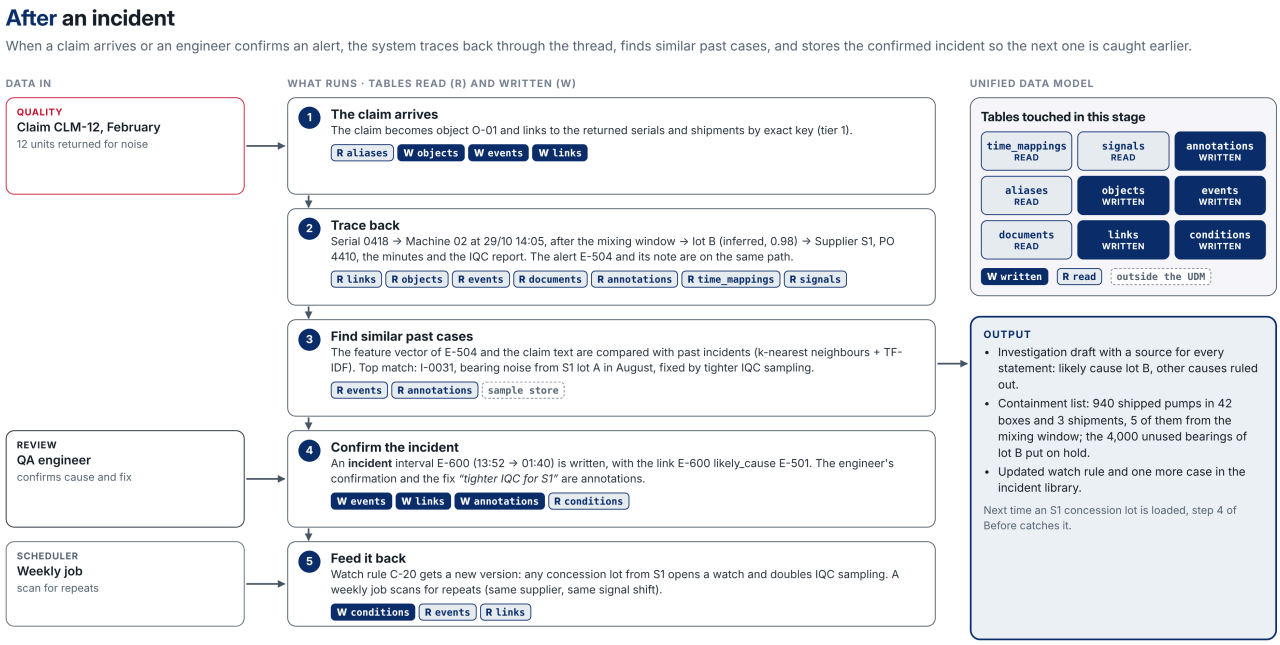


Figure 10. After. The confirmed incident and its fix go back into the conditions, so the next S1 concession lot opens a watch with tighter IQC.

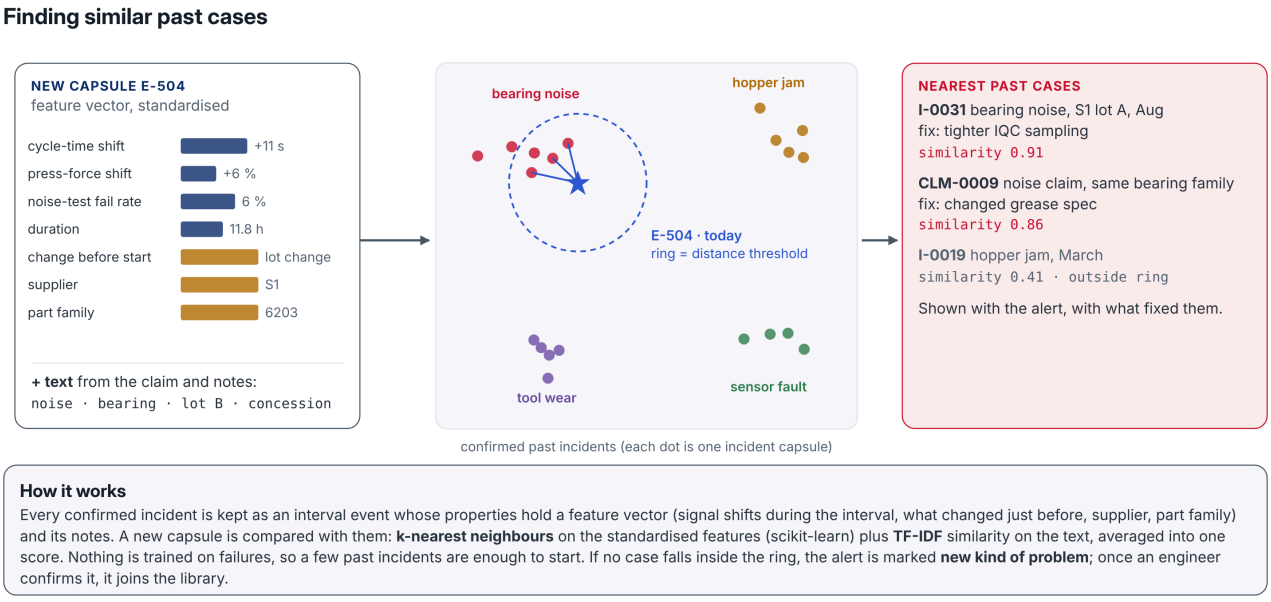


Figure 11. Similar past cases with simple methods: a feature vector per incident, k-nearest neighbours and TF-IDF. The ring is the distance threshold; outside it, the case is new.

Keeping alerts and reviews manageable

Too many alerts, and too many items to review, are the most likely way for this system to fail in a plant. The rules are:

- **An alert cap per line and shift.** Each line sets how many alerts people will act on in a shift (for example five). Only the highest-ranked alerts up to that number reach the dashboard and the review queue; the rest are logged and appear in the daily digest.
- **New detectors start in shadow mode.** A new or changed condition writes its events without alerting anyone. It goes live only when its precision on that period, measured against what engineers confirmed, is acceptable to the line's quality engineer.
- **Every review item has an owner and a target time.** Aliases go to the data team, tier 4 links to the engineer of the department concerned, alerts to the shift lead and QA. Items are batched into one daily list per person. Tier 5 suggestions never block anything and expire if nobody looks at them.
- **The review load is measured.** The Connections screen shows items created and closed per day and per owner, so the team sees when a rule creates more work than people can handle and tightens it.
- **The agent investigates ranked alerts only.** Automatic investigations run for alerts that pass the cap, for claims and for confirmed incidents, not for every anomaly event.

Feedback and deep learning

Every decision in the review queue is an annotation, so each condition has a measured precision: how many of its alerts were confirmed. Limits are tuned from that, baseline models are retrained weekly on recent normal production, and confirmed incidents grow the library for similar cases. **Deep learning is not needed for the demo or the first pilot.** It becomes worth adding where the data supports it: high-rate waveforms such as injection pressure, where commercial monitors already use neural networks (FANUC and Preferred Networks' AI Backflow Monitor [13]), or once enough incidents are labelled to learn embeddings of signal windows [10, 11]. The tables do not change: a neural detector is one more condition writing events.

TECH Apache Flink for stream rules, scikit-learn and LightGBM for baselines, PCA, Isolation Forest and k-nearest neighbours, pgvector for text similarity, Claude for 4M extraction

6 The AI agent and its tools

The agent never queries databases directly. It plans the steps, calls tools from the MCP server, reads what comes back and writes an answer with a source for every number. All tools are read-only and pass through the same permissions and audit log as a human user.

GROUP	TOOL	RETURNS
Graph	`find(name)`	The object or event a name refers to, resolved through aliases
	`neighbours(id, relation, at_time)`	Linked records with tier, status, score, valid time
	`trace(id, direction, at_time)`	Forward or backward path, using only links valid at that time
Catalog	`list(domain)`	Datasets and signals for a domain
	`describe(dataset or signal)`	Fields, owner, freshness, where it lives (lakehouse or source)
Query	`events(object, window, type)`	Events in a time window, including overlap with a given interval
	`samples(signal, window)`	Signal samples on plant time, downsampled if needed
	`query(dataset, filter)`	One SQL call to the lakehouse or a federated source; KPIs from Gold
	`search_docs(text, filters)`	Document chunks by keyword and meaning, filtered by department, type, date, line or process; each hit says whether it is also linked to the object asked about
	`similar(event_id)`	Closest past incidents with their fixes and a similarity score
Source	`get_file(doc_id)`	Document chunks, or a file such as a drawing from PLM
	`run_python(code)`	Analysis on data already returned (trend, comparison by lot) in a sandbox with no network

The agent may only treat confirmed and inferred links as evidence, and must say which one it used. suggested links can be mentioned only as leads to check.

How the agent answers a question

Most questions are answered by querying the department databases, not by walking the graph one object at a time. The graph and the links decide *what* to query; the databases return the detail and the numbers. Three patterns cover almost every question:

PATTERN	EXAMPLE QUESTION	WHAT THE AGENT DOES
Query the databases	"Noise-test fail rate by supplier lot in October"	`list` and `describe` find the datasets; one `query` joins the Quality and Warehouse stores. It works because steps 2 and 4 put the same IDs on both sides, and the lot-to-serial links act as the join table.
Scope with the graph, then query	"Why did claim CLM-12 happen?"	`trace` gives the scope: serial 0418 → Machine 02 at 14:05 → lot B → Supplier S1. Then ordinary queries fetch the detail: `samples` for SIG-17 around 14:05, IQC measurements for lot B, order detail from ERP.

PATTERN	EXAMPLE QUESTION	WHAT THE AGENT DOES
Search the documents	"What do our SOPs and past reports say about bearing noise after press-fit?"	`search_docs` runs keyword and vector search over all chunks, filtered by the context already known (department Quality and Production, process press-fit, last three years). No link is needed.

When a question starts from an object, documents linked to it come first, then search results within the same filters. The two are labelled differently. Quoting what a document says is always allowed ("the press-fit SOP requires a force check every 50 parts"). Claiming that a document is about a specific object needs a link: a search hit that seems to concern lot B is shown as a lead, the same rule as tier 5, and becomes a link only when an engineer confirms it.

AI agent for in-depth questions

A user asks in plain language. The agent resolves names through the aliases table ("M02" becomes Machine 02), splits the question into steps, calls the tools above and runs any calculation in the sandbox. Before it answers, it checks that every statement cites a record: tier 4–5 links appear only as leads, and missing data is named instead of guessed. The answer comes with a chart, the sources and suggested next checks. A follow-up question or "make a report" continues from the same evidence.

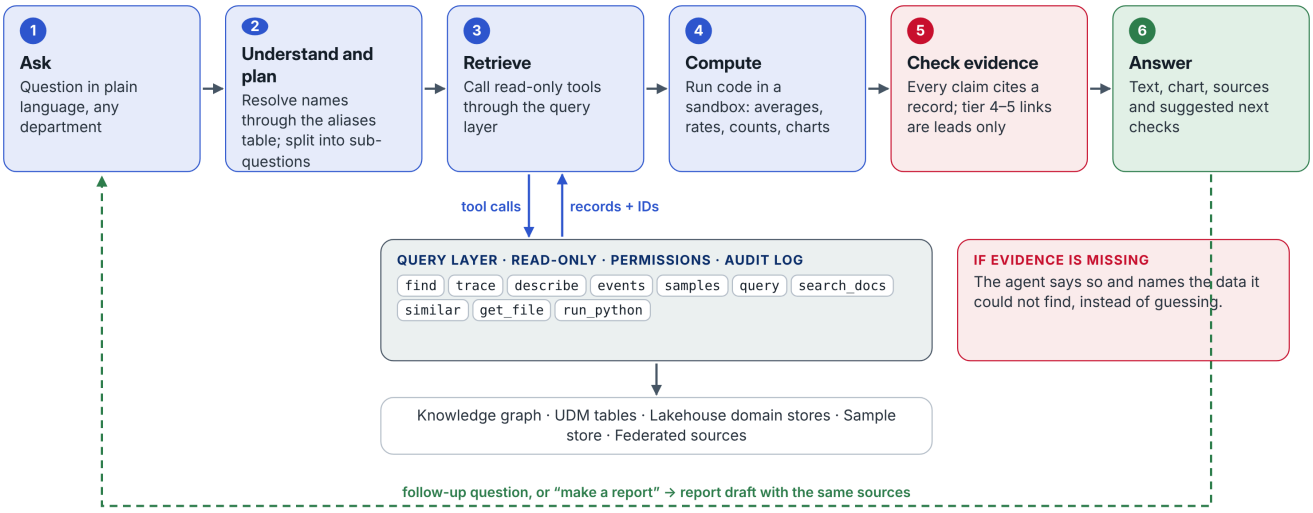


Figure 12. Answering an in-depth question. Steps 3 and 4 run through the query layer, so the agent sees only what the user is allowed to see, and every read is logged.

AI agent for automatic investigations and reports

Event-driven. A new change event, an alert that passed the line's cap, a claim or confirmed incident, a link waiting for review or a late data source puts an event on the bus. The agent gathers the context, works out the impact (affected serials, stock, shipments, similar past cases) and drafts an investigation brief with suggested checks. The brief goes to the dashboard and to the owner; an engineer approves any action. **Scheduled.** The scheduler starts fixed tasks: a summary every shift, a daily data-health and review-queue digest, a weekly supplier quality digest and scan for repeats, and a monthly KPI report draft.

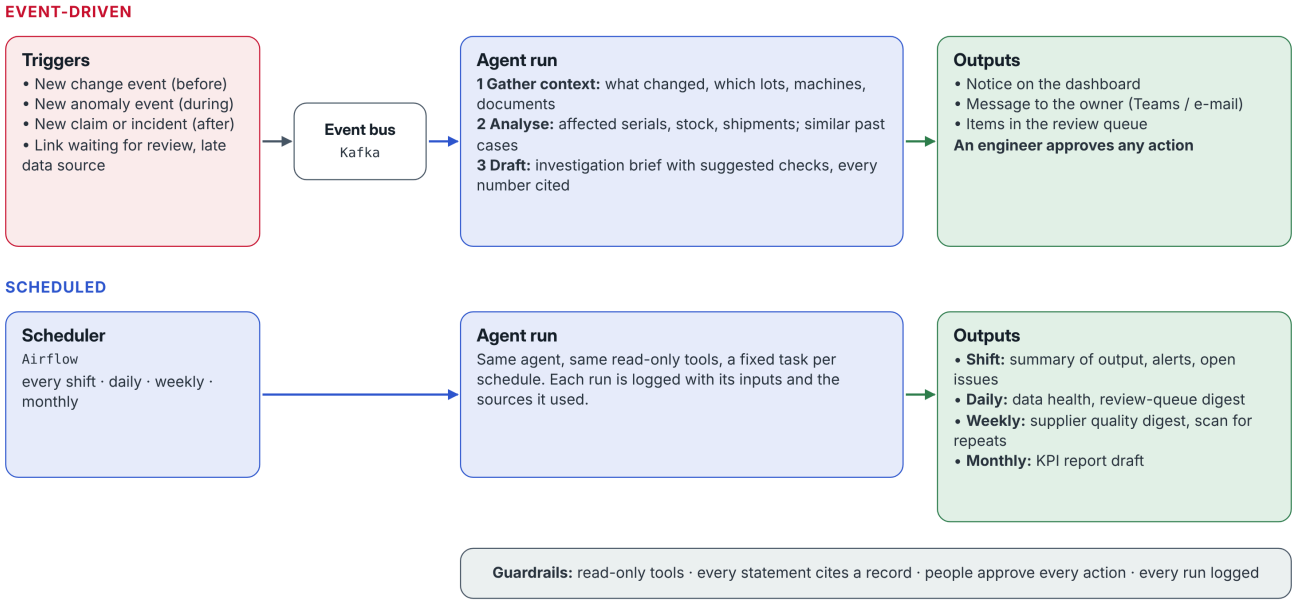


Figure 13. Automatic runs. Both lanes use the same agent and the same read-only tools; nothing is changed in any system without a person approving it.

TECH Kafka for events, Airflow for schedules, the MCP server of the query layer, Claude or an on-premises model

7 Tracing a customer claim three months later

The claim arrives in February; the pumps were made on 29 October and passed the end-of-line noise test, so this is a latent defect. Because every record was linked when it arrived, the October picture is still complete.

#	WHAT HAPPENS	WHERE IN THE ARCHITECTURE
1	Claim CLM-12 is logged: 12 MH-20 pumps returned for noise, including serial 0418.	QMS → steps 1–4; the claim links to the serials by exact key (tier 1).
2	The user opens the claim. `trace(0418, back)` shows serial 0418 pressed on Machine 02 on 29/10 at 14:05, with Bearing 6203 from lot B.	Graph tools. 14:05 is inside the `fed_into` interval and after the 13:52–13:56 mixing window, so the lot B link is inferred with score 0.98.
3	Machine 02 that afternoon: cycle time rose from 30 s to 42 s right after the 13:52 refill with lot B, and the noise-test fail rate from 0.5 % to 6 %. The anomaly E-504 was raised at 13:54 and noted "in spec, keep running (shortage)".	`events` and `samples`. The 13:52 match is correct only because step 3 removed the PLC's +38 s offset.
4	Lot B's documents: the minutes of 22/10 accepting it as a concession, the IQC report with Cpk 0.92, near the upper limit, and drawing 6203.	`neighbours` and `get_file`; the minutes link was attached by tier 4 after review.
5	Similar past cases: I-0031, bearing noise from S1 lot A in August, fixed by tighter IQC sampling; an older defect report about bearing noise is shown as a lead only.	`similar(E-504)`; tier 5 suggestion for the old report, marked as unconfirmed.
6	The agent drafts: "Likely cause: bearing lot B, a concession lot with Cpk 0.92. No maintenance, operator change or climate event on Machine 02 in the window. I-0031 was fixed by tighter IQC sampling." Each sentence cites its records.	AI agent with `events` (time slice), `query` and `run_python` (comparison by lot).
7	Containment: lot B went into 1,000 pumps; 940 passed the noise test and shipped in 42 boxes and 3 shipments. The 5 serials from the mixing window are listed as "lot A or B". The 4,000 unused bearings of lot B are put on hold.	Forward trace: lot → `processed_on` → `packed_in` → `shipped_in`.
8	The QA engineer confirms: incident E-600, cause lot B, fix "tighter IQC for S1". The watch rule for S1 concession lots gets a new version.	`events`, `links`, `annotations`, `conditions` (section 5, After).

Without the thread, steps 2 to 7 mean emails to production, quality, purchasing and logistics, each answering in its own names and clock. With the before and during stages running, the same facts were already on the dashboard on 29 October, two minutes after lot B was loaded.

8 Demo slice and target architecture

The demo shows what the system can do on realistic data. The architecture shows how it scales. The demo runs the same logic as the target, on a smaller stack.

PART	IN THE DEMO	AT PLANT SCALE
Sources	Sample files built to be messy: different machine names, PLC clock offset, a PDF design file, minutes in Vietnamese, a defect report without part codes	Live systems

PART	IN THE DEMO	AT PLANT SCALE
Ingestion, Bronze	Files replayed in time order	Connectors, Kafka, RabbitMQ, MinIO
Steps 1–4	One Python script, tiers 1–4	Flink jobs, tier 5 added
Registries	YAML files; aliases, signals, conditions and time_mappings as tables	Same, with review screen and Git history
Data model	The nine tables in PostgreSQL; samples in TimescaleDB	Same, with pgvector and Iceberg domain stores
Views	Queries straight on the tables	Redis, lakehouse, Neo4j
Early warning	LLM 4M extraction, limits, CUSUM, baseline model, similar cases on a simulated incident library	All methods in section 5, tuned per line
Query layer, AI	FastAPI + MCP server + agent	Same, with Keycloak and audit log
Web app	Plant, Product, Assistant; Reports prefilled	All five screens

9 Technology summary

LAYER	TECHNOLOGY	WHY
Edge	OPC UA, Mosquitto (MQTT)	Factory standards; store-and-forward when offline
Database capture	Debezium	Reads MES/WMS changes from the log without loading the source
Batch pulls	Airflow	Schedules API and CSV pulls, nightly exports, replays
Event bus	Apache Kafka	Ordered, durable, many readers
Document intake	FastAPI, RabbitMQ	Simple upload door and per-file job queue
Raw storage	MinIO, Apache Iceberg	On-premises object storage; tables that can be replayed and versioned
Processing	Apache Flink	Stateful stream joins for identity, time and linking
Document parsing	Docling	Layout-aware PDF and scan parsing into chunks
LLM	Claude	4M change and code extraction, mapping suggestions, the agent; replaceable by an on-premises model
Data model	PostgreSQL, TimescaleDB, pgvector	Nine tables, signal samples, embeddings and full-text search in one database
Live state	Redis	Second-level reads for plant screens
Lakehouse	Iceberg, dbt, Trino	Domain stores, tested KPI models, SQL over large tables and federated sources
Graph	Neo4j	Fast multi-hop traces and containment queries
Query layer	FastAPI, MCP server, Keycloak	One API for app and AI; roles and sign-in
Early warning	Flink, scikit-learn, LightGBM, pgvector	Stream rules, baseline models, PCA, Isolation Forest, similar past cases
Agent automation	Kafka, Airflow	Event-driven and scheduled agent runs
Monitoring, alerts	Prometheus, Grafana	Pipeline health; dashboards and alert rules
Configuration	Git (YAML), PostgreSQL	Every mapping and rule change reviewed and reversible
Web app	React, React Flow	Interactive graph screens

10 Open decisions

- **Where the LLM runs.** If plant data may not leave the site, the agent and extraction move to an open model served on-premises. The architecture does not change; the answer quality needs testing.
- **How link and alert quality is measured.** Each tier and each condition needs a precision check on labelled samples before its output is trusted, and tier 3 window sizes need tuning per station.

- **How much incident history exists.** Similar-case search needs past incidents with a cause and a fix. If old defect reports are not structured, the first months run on before and during only while the library grows.
- **When to add deep learning.** Only where simple methods miss problems that matter and enough data exists, starting with high-rate waveforms.
- **Access rules.** Which roles may see HR, cost and supplier data, and whether the agent inherits the asking user's permissions (recommended) or has its own.
- **Scale of processing.** Flink is sized for many lines and high-rate sensors. A single plant pilot may run steps 1–4 as plain Python services first.

11 References

1. Page, E. S. (1954). Continuous inspection schemes. *Biometrika*, 41(1/2), 100–115.
2. Montgomery, D. C. (2019). *Introduction to Statistical Quality Control* (8th ed.). Wiley.
3. MacGregor, J. F., & Kourti, T. (1995). Statistical process control of multivariate processes. *Control Engineering Practice*, 3(3), 403–414.
4. Liu, F. T., Ting, K. M., & Zhou, Z.-H. (2008). Isolation forest. *IEEE International Conference on Data Mining (ICDM)*.
5. Chen, T., & Guestrin, C. (2016). XGBoost: a scalable tree boosting system. *KDD*.
6. Ke, G., et al. (2017). LightGBM: a highly efficient gradient boosting decision tree. *NeurIPS*.
7. Stewart, M., Hodkiewicz, M., & Li, S. (2023). Large language models for failure mode classification: an investigation. arXiv:2309.08181.
8. Seeq. Data in Seeq: signals, conditions and capsules. support.seeq.com/latest/cloud/data-in-seeq.
9. Pedregosa, F., et al. (2011). Scikit-learn: machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
10. Yue, Z., et al. (2022). TS2Vec: towards universal representation of time series. *AAAI*.
11. Malhotra, P., et al. (2016). LSTM-based encoder-decoder for multi-sensor anomaly detection. *ICML Anomaly Detection Workshop*.
12. Reimers, N., & Gurevych, I. (2019). Sentence-BERT: sentence embeddings using Siamese BERT-networks. *EMNLP*. (For text similarity, if TF-IDF is not enough.)
13. FANUC and Preferred Networks. AI Backflow Monitor for ROBOSHOT injection moulding machines (2018), an industrial example of deep learning on machine waveforms.

A How the two team drafts were merged

IDEA	FROM	WHERE IT LIVES NOW
Read-only pull connectors, one per source type	Pipeline draft	Ingestion, systems door
Common POST endpoint + RabbitMQ	Report draft	Ingestion, documents door
Raw copy and replay	Pipeline draft	Bronze
Normalise time, names, IDs	Both	Steps 1–3, each with its own registry
Five-tier linking ladder, confirmed / inferred / suggested	Report draft	Step 4
Chunk documents, LLM classifies, engineer reviews	Report draft	Step 1 document lane and tier 4
Static documents on nodes, logs on edges	Report draft	Documents attach to objects through links; events attach to objects and time; drawn on edges in Plant view
Redis, MinIO	Report draft	Redis for live state; MinIO for Bronze and files
Live state, lakehouse, knowledge graph	Pipeline draft	Read-optimised views
Query layer with permissions; federated path	Pipeline draft	Query layer and catalog
AI suggests, engineer approves, versioned	Both	Control plane
AI agent through MCP tools	Both	Applications